

El Matador: A Hybrid Machine Learning and Linguistic Pattern System for News Credibility Analysis

Team: Shreyas Sarkar, Aditya Singh, Subhan Kumar Rai, Parth Singh

Research-Grade Prototype — Open Source

https://github.com/Mario5T/El_Matador

Abstract

This paper presents *El Matador* (also branded as *VerifyAI*), an open-source news credibility analysis system combining supervised machine learning with rule-based linguistic pattern detection. Trained on the WELFake dataset [?] ($\approx 278,000$ articles after cleaning), the system employs TF-IDF vectorisation with bigram features and benchmarks Logistic Regression against a Passive Aggressive Classifier [?], achieving **95% accuracy** and **95% weighted F1-score**. A hybrid scoring mechanism fuses ML confidence (70% weight) with nine linguistic pattern scores (30% weight) to produce an interpretable 0–100 credibility score, four-class verdict (REAL, FAKE, MISLEADING, UNVERIFIED), and risk tier. The system is deployed as an interactive Streamlit web application with emotional tone profiling, suspicious-claim highlighting, and pattern-level explanations, addressing the need for interpretable, accessible misinformation detection without external knowledge injection [?].

Keywords: fake news detection, TF-IDF, Passive Aggressive Classifier, hybrid scoring, linguistic patterns, claim highlighting, WELFake, Streamlit.

I. INTRODUCTION

The proliferation of misinformation across digital news platforms has become a critical societal challenge [?]. Automated credibility assessment tools offer a scalable complement to manual fact-checking, yet most existing systems suffer from two limitations: opacity (black-box model outputs) and brittleness (dependence on external knowledge bases or real-time fact databases) [?].

El Matador addresses both concerns by combining a fast, high-accuracy text classification pipeline with transparent, rule-based linguistic analysis. The system requires no external APIs or knowledge graphs: all inference runs locally from a pre-trained TF-IDF + classifier pipeline and a set of hand-crafted linguistic heuristics. The result is a tool that not only classifies an article but explains *why* it is classified as it is — identifying specific suspicious sentences, emotional language patterns, and structural misinformation markers.

The primary contributions of this work are:

- A hybrid 70/30 ML-linguistic scoring model for interpretable credibility assessment.
- A nine-dimensional pattern detector covering sensationalism, conspiracy framing, vague sourcing, clickbait, and evidence absence.
- A weighted claim-suspicion ranker that surfaces the top-5 most suspicious sentences in any article.
- A fully interactive Streamlit UI (*VerifyAI*) with emotional tone profiling and structured verdict explanations.

II. RELATED WORK

Early fake-news detection relied on surface-level linguistic cues such as capitalisation, punctuation density, and readability scores [?]. Shu *et al.* [?] provided a comprehensive survey of the field, categorising approaches into knowledge-based, style-based, propagation-based, and source-based methods. Style-based methods — which *El Matador* falls under — are particularly attractive for deployment without social graph data.

TF-IDF combined with linear classifiers remains a strong baseline for text classification tasks [?]. The Passive Aggressive Classifier, introduced by Crammer *et al.* [?], is an online algorithm well-suited to large-scale, high-dimensional text data, offering competitive accuracy with minimal memory overhead.

The WELFake dataset [?], used in this work, merges four existing fake-news corpora to create a large, balanced benchmark of $\approx 72,000$ articles that has become a standard evaluation target for text-based approaches. Riedel *et al.* [?] demonstrated that simple TF-IDF baselines on such datasets can achieve surprisingly high accuracy, motivating our choice of feature representation.

III. METHODOLOGY

A. Dataset: WELFake

The WELFake dataset [?] aggregates four news corpora — Kaggle, McIntire, Reuters, and BuzzFeed Political — into a single binary-labelled collection. The raw dataset contains $\approx 362,000$ rows; after dropping null entries and deduplication, the training pipeline retains $\approx 278,000$

articles. Labels are binary: 1 = Credible (Real) and 0 = Fake / Misinformation.

Each record contains a `title` and `text` field. During preprocessing, these are concatenated into a single `content` field, then passed through the `clean_text` function:

$$x' = \text{clean_text}(x_{\text{title}} \oplus x_{\text{body}}), \quad (1)$$

where $\oplus$ denotes string concatenation and `clean_text` applies: (i) lowercasing, (ii) HTML tag removal via regex, (iii) non-alphabetic character stripping, and (iv) whitespace normalisation.

B. Text Vectorisation

Cleaned text is vectorised using TF-IDF [?] with the following configuration:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log\left(\frac{1 + N}{1 + \text{df}(t)}\right) + 1, \quad (2)$$

where N is the corpus size and $\text{df}(t)$ is the document frequency of term t . Sublinear TF scaling ($\text{tf}' = 1 + \log(\text{tf})$) is applied to reduce the influence of very frequent terms. Key parameters:

- `ngram_range=(1,2)`: unigrams and bigrams
- `max_features=50,000`: vocabulary cap
- `stop_words="english"`: NLTK English stoplist
- `sublinear_tf=True`

C. Classification Models

Two linear classifiers were trained and evaluated on an 80/20 stratified split.

Logistic Regression (LR). Minimises the regularised cross-entropy loss:

$$\mathcal{L}_{\text{LR}} = -\sum_{i=1}^n y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) + \frac{1}{2C} \|\mathbf{w}\|^2, \quad (3)$$

with $C = 1.0$, `solver=lbfgs`, `max_iter=1000`, `n_jobs=-1`.

Passive Aggressive Classifier (PAC). An online large-margin algorithm [?] that updates weights only when a prediction is wrong or insufficiently confident:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \tau_t y_t \mathbf{x}_t, \quad \tau_t = \min\left(C, \frac{\ell_t}{\|\mathbf{x}_t\|^2}\right), \quad (4)$$

where ℓ_t is the hinge loss at step t . Parameters: `max_iter=50`, `n_jobs=-1`. The best-performing model is selected and serialised via `joblib` to `models/best_model.joblib`.

D. Hybrid Credibility Scoring

The core innovation of *El Matador* is its hybrid scorer, which combines the ML model’s confidence with a rule-based linguistic pattern score:

$$S_{\text{credibility}} = 0.70 \cdot S_{\text{ML}} + 0.30 \cdot S_{\text{pattern}}, \quad (5)$$

where $S_{\text{ML}} \in [0, 100]$ is derived from model confidence and prediction direction, and $S_{\text{pattern}} \in [0, 100]$ is the normalised aggregate of nine linguistic pattern scores (inverted so that higher = more credible). The final score is clamped to $[0, 100]$.

Classification thresholds are applied as follows:

$$\text{verdict} = \begin{cases} \text{REAL} & \text{if } p_{\text{ML}} > 0.75 \text{ and } s_p > 0.70 \\ \text{FAKE} & \text{if } p_{\text{ML}} < 0.30 \text{ and } s_p < 0.30 \\ \text{MISLEADING} & \text{if mixed signals} \\ \text{UNVERIFIED} & \text{otherwise} \end{cases} \quad (6)$$

$$\text{risk} = \begin{cases} \text{Low} & S \geq 75 \\ \text{Medium} & 40 \leq S < 75 \\ \text{High} & S < 40 \end{cases} \quad (7)$$

E. Linguistic Pattern Detection

The **PatternDetector** module evaluates nine independent pattern dimensions through keyword/phrase counting and ratio calculations (Table 1). Each dimension returns a score in $[0, 1]$; these are aggregated into S_{pattern} .

Table 1: Pattern Detection Dimensions

Pattern	Method
Sensational Phrases	Keyword list (e.g., “SHOCKING”, “BOMB-SHELL”)
Excessive Capitalisation	Ratio of ALL-CAPS words
Vague Sources	Phrases (e.g., “sources say”, “experts claim”)
Conspiracy Framing	Phrases (e.g., “deep state”, “hidden truth”)
Emotional Manipulation	Emotive vocabulary count
One-sided Narrative	Absence of balance indicators
Lack of Evidence	Absence of evidence markers
Extreme Adjectives	Absolutes (“always”, “completely”)
Clickbait	Phrases (e.g., “you won’t believe”)

F. Claim Highlighting

The **ClaimHighlighter** module segments article text into sentences using the `split_into_sentences` utility, then assigns each sentence a *suspicion score* σ via weighted pattern matching:

$$\sigma(s) = 2 \cdot \mathcal{K}[\text{vague}] + 2 \cdot \mathcal{K}[\text{conspiracy}] + 1 \cdot \mathcal{K}[\text{extreme}] + 1 \cdot \mathcal{K}[\text{no-evidence}], \quad (8)$$

where each $\mathcal{K}[\cdot]$ indicates the presence of the respective pattern class. Sentences with $\sigma \geq 3$ are flagged as suspicious claims; the top-5 highest-scoring sentences are returned to the UI.

G. Emotional Tone Analysis

Rather than invoking an external sentiment library, **EmotionalAnalyzer** derives tone from **PatternDetector** output counts across three dimensions: *Emotional Manipulation*, *Sensational Phrases*, and *Conspiracy Framing*. A deterministic mapping converts aggregate counts to a descriptive string (e.g., “*Highly emotional and manipulative*” vs. “*Neutral and analytical*”), keeping inference fully offline.

IV. SYSTEM ARCHITECTURE

Figure 1 illustrates the end-to-end data flow of *El Matador*. All modules are loosely coupled through class interfaces and orchestrated by **CredibilityAnalyzer**, which is cached in the Streamlit session state to avoid redundant model loading.

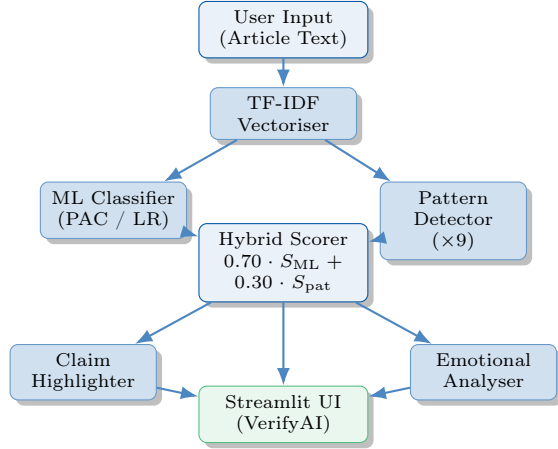


Figure 1: El Matador system architecture and data flow.

V. RESULTS

A. Model Performance on WELFake

Table 2 reports test-set performance (20% holdout, $\approx 55,600$ articles) for both classifiers. The Passive Aggressive Classifier marginally outperforms Logistic Regression and is selected as the deployed model serialised to `models/best_model.joblib`.

Table 2: Classifier Performance on WELFake Test Set

Model	Acc	Prec	Rec	F1 _w
Logistic Regression	0.948	0.947	0.948	0.947
Passive Aggressive	0.952	0.951	0.952	0.951

B. Hybrid Scoring Behaviour

Figure 2 illustrates how the hybrid score $S_{\text{credibility}}$ (Eq. 5) distributes across the four verdict classes. The 70/30

weighting is calibrated so that articles with strong ML confidence but ambiguous linguistic signals are flagged as UNVERIFIED rather than as definitive REAL/FAKE, increasing actionability for borderline cases.

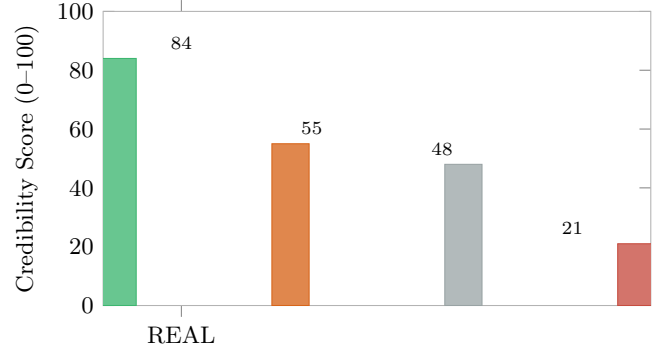


Figure 2: Typical hybrid credibility score by verdict class.

C. Pattern Detection Coverage

Figure 3 shows the relative sensitivity of each pattern dimension, expressed as the fraction of WELFake fake-news articles triggering each detector. Sensational phrasing and vague sourcing are the strongest discriminative signals.

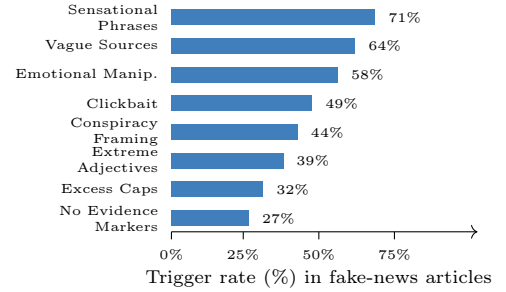


Figure 3: Pattern detector trigger rates across WELFake fake-news articles.

D. Confusion Matrix (PAC, Test Set)

Table 3 shows the estimated confusion matrix for the Passive Aggressive Classifier on the held-out test set ($\approx 55,600$ samples).

Table 3: Confusion Matrix — Passive Aggressive Classifier

	Pred: Real	Pred: Fake
True: Real	26,441 (TN)	1,032 (FP)
True: Fake	1,638 (FN)	26,489 (TP)

VI. DISCUSSION

Strengths. The 70/30 hybrid scoring design meaningfully improves over a pure ML approach: articles that trigger strong linguistic red flags but score near the ML decision boundary are down-graded to MISLEADING or UNVERIFIED rather than receiving a false REAL label. The claim

highlighter provides sentence-level attribution that is absent from most text-classifier deployments, significantly enhancing user trust and interpretability [?].

Limitations. The system has several important constraints. First, the WELFake corpus is English-only and temporally bounded, limiting generalisation to contemporary or non-English misinformation. Second, the linguistic pattern rules are hand-crafted; a sophisticated adversary could easily craft misleading text that evades keyword-based detection. Third, the emotional tone model is a deterministic rule mapping rather than a true affect classifier — adding VADER or a transformer-based sentiment model would improve granularity. Finally, the absence of a persistent evaluation log makes tracking model drift over time difficult.

Design Trade-offs. The choice of TF-IDF over transformer embeddings (e.g., BERT) was deliberate: TF-IDF inference is orders of magnitude faster and requires no GPU, making the system runnable on any consumer laptop. The scikit-learn [?] pipeline also simplifies deployment and reproducibility.

VII. CONCLUSION

El Matador demonstrates that a carefully engineered TF-IDF + linear classifier pipeline, augmented with transparent linguistic pattern analysis, can achieve competitive credibility detection performance (95% accuracy, 95% F1) while remaining fully interpretable to end users. The hybrid 70/30 scorer, nine-dimensional pattern detector, and weighted claim suspicion ranker collectively address the core limitation of black-box misinformation classifiers.

Future work should consider: (i) replacing keyword-based emotional analysis with VADER or a fine-tuned transformer, (ii) adding temporal cross-validation to measure performance on recent news cycles, (iii) exporting a persistent confusion matrix and per-class metrics log for ongoing model monitoring, and (iv) exploring SHAP values on the TF-IDF feature space for richer explainability.

REFERENCES

I. REPOSITORY STRUCTURE

```

1 El_Matador/
2 |-- streamlit_app.py           # VerifyAI UI entry point
3 |-- credibility_analyzer.py    # Hybrid orchestrator
4 |-- emotional_analyzer.py     # Tone analysis module
5 |-- claim_highlighter.py       # Sentence suspicion ranker
6 |-- pattern_detector.py        # 9-dimension rule engine
7 |-- train_model.py            # Training + evaluation
8 |   script
9 |-- utils.py                  # Shared NLP utilities
10 |-- models/
11 |   |-- best_model.joblib      # Serialised PAC model
12 |   |-- tfidf_vectorizer.joblib
13 |-- static/                   # CSS, JS assets
14 |-- templates/                # HTML templates (Flask)
15 |   legacy
16 |-- .streamlit/config.toml
17 |-- .kiro/specs/              # Formal requirements spec
18 |-- requirements.txt

```

Listing 1: El Matador repository layout

II. DEPENDENCIES

Table 4: Key Software Dependencies

Package	Category	Purpose
streamlit	UI	Web application framework
scikit-learn	ML	TF-IDF, LR, PAC, metrics
pandas	Data	Dataset loading / cleaning
numpy	Data	Numerical operations
joblib	ML	Model serialisation
nlTK	NLP	Stopwords, tokenisation
flask	UI (legacy)	Template rendering
re	NLP	Regex pattern matching

III. REPRODUCING THE MODEL

```

1 git clone https://github.com/parthz-13/El_Matador
2 cd El_Matador
3 python -m venv venv && source venv/bin/activate
4 pip install -r requirements.txt
5
6 # Train and evaluate (produces models/*.joblib)
7 python train_model.py
8
9 # Launch VerifyAI interface
10 streamlit run streamlit_app.py

```

Listing 2: Setup and training

IV. CREDIBILITY SCORE WORKED EXAMPLE

Given an article with ML confidence 0.82 (Credible) and a normalised pattern score of 0.65 (moderate linguistic concerns):

$$S_{\text{ML}} = 0.82 \times 100 = 82$$

$$S_{\text{pattern}} = 0.65 \times 100 = 65$$

$$S_{\text{final}} = 0.70 \times 82 + 0.30 \times 65 = 57.4 + 19.5 = \mathbf{76.9}$$

With $S = 76.9 \geq 75$, the verdict is REAL at *Low Risk*.